K.R. MANGALAM UNIVERSITY

Unit–2 Assignment: Linear Data Structures

SUBMITTED BY- Sree Parvati

ROLL NO - 2501730236

COURSE - Data Structures

PROGRAMME - B. Tech CSE (AI & ML)

SECTION - A

SUBMITTED TO - Mrs. Neetu Chauhan

Task 1: Dynamic Array

Goal

To understand how dynamic arrays automatically grow when they become full.

Implementation

A Dynamic Array class was implemented with:

- Capacity (maximum elements it can hold)
- Size (current number of elements)
- Internal array for storing elements

append(x) adds elements and doubles capacity when full, while pop() removes the last element. The array is displayed using __str__().

Observation

- Initial capacity: 2
- Resizing observed:
  2 → 4 → 8 → 16
- After performing 3 pop () operations, the last elements were removed correctly and array updated.

Time Complexity Analysis (Amortized Analysis)

- In most cases, append () simply adds an element at the end → **O(1)**
- When the array becomes full:
  - A new array is created
  - All elements are copied → **O(n)**

However, resizing does not happen every time. Since capacity doubles, the number of costly operations is small compared to total insertions.

Conclusion:
Most appends are **O(1)**, only a few are **O(n)** due to resizing.
So, overall **amortized time complexity of append() is O(1)**.

Task 2: Linked List

Goal

To implement a dynamic structure where insertion and deletion do not require shifting of elements.

(A) Singly Linked List

Operations:

- Insert at beginning → O(1)
- Insert at end → O(n)
- Delete by value → O(n)
- Traverse → O(n)

Observation:
All operations worked correctly. Insertions and deletion updated links properly, and traversal showed elements in correct order.

(B) Doubly Linked List

Includes both next and prev pointers.

Operations:

- Insert after node → O(n)

- Delete at position (0-based) → O(n)

Observation:
Forward and backward links were maintained correctly after insertion and deletion.

Task 3: Stack & Queue

(A) Stack (LIFO)

Implemented using linked list (operations at head).

Time Complexity (Stack)

Operation	Time Complexity	Stack
push(x)	O(1)	Insert at head, no traversal
pop()	O(1)	Remove from head directly
peek()	O(1)	Access top element

(B) Queue (FIFO)

Implemented using linked list with front and rear pointers.

Time Complexity (Queue)

Operation	Time Complexity	Explanation
enqueue(x)	O(1)	Insert at rear using pointer
dequeue()	O(1)	Remove from front
front()	O(1)	Access from element

Complexity Discussion

Both stack and queue operations are efficient because:

- No traversal is required
- Direct access using pointers is used

Hence, all operations run in constant time O(1).

Task 4: Balanced Parentheses Checker

Goal

To check whether an expression has balanced brackets using stack.

Logic

- Push opening brackets
- Pop and match closing brackets
- If mismatch or stack not empty → Not balanced

Test Cases

Input	Output
([])	Balanced
([)]	Not Balanced
(((	Not Balanced
“”	Balanced

Time Complexity

- Single traversal of string → **O(n)**

Conclusion

This assignment provided a clear understanding of linear data structures and their internal working. Dynamic arrays demonstrated how resizing helps in efficient memory usage, while linked lists showed flexibility in insertion and deletion. Stack and queue implementations highlighted how abstract data types can be built using linked lists with constant time operations. The balanced parentheses checker demonstrated a practical application of stacks. Overall, the assignment strengthened both conceptual knowledge and coding skills.

References

- GeeksforGeeks
- W3Schools
- Notes from the lecture